

LAB 6. KẾT NỐI CSDL & REPOSITORY PATTERN

Ở các buổi trước, FakeProductRepository đã phục vụ chúng ta rất tốt cho việc xây dựng giao diện và logic. Nhưng giờ là lúc để nó "nghỉ hưu". Chúng ta sẽ sử dụng một công nghệ cực mạnh là **Entity Framework Core (EF Core)** để làm cầu nối giữa ứng dụng C# và cơ sở dữ liệu quan hệ, đồng thời chứng minh sức mạnh của **Repository Pattern** mà chúng ta đã dày công thiết lập.

MỤC TIÊU BÀI HỌC

Kết thúc buổi học này, bạn sẽ có khả năng:

- **Hiểu rõ vai trò của ORM** (Object-Relational Mapping) và EF Core.
- **Cài đặt và cấu hình** Entity Framework Core trong một dự án.
- **Tạo một DbContext** để quản lý phiên làm việc với cơ sở dữ liệu.
- **Triển khai một Repository cụ thể** (EFProductRepository) để truy vấn dữ liệu từ CSDL bằng LINQ.
- **Chuyển đổi hoàn toàn** nguồn dữ liệu của ứng dụng chỉ bằng cách thay đổi **một dòng code** trong DI Container, minh chứng cho sức mạnh của kiến trúc đã xây dựng.

YÊU CẦU KIẾN THỨC

- Đã hoàn thành bài thực hành số 5.
- Hiểu biết cơ bản về cơ sở dữ liệu quan hệ (bảng, cột, khóa chính).

CÔNG CỤ CẦN THIẾT

- Solution SportsStore đã hoàn thành ở buổi 5.
- Một hệ quản trị CSDL. **SQL Server Express LocalDB** là lựa chọn đơn giản nhất vì nó thường được cài đặt sẵn cùng Visual Studio.

6.1. Tóm tắt lý thuyết

6.1.1. ORM và Entity Framework Core là gì?

Hãy tưởng tượng bạn (lập trình viên C#) nói "ngôn ngữ đối tượng" (Objects, Classes, Properties), còn cơ sở dữ liệu (ví dụ SQL Server) lại nói "ngôn ngữ quan hệ" (Tables,

Columns, Rows). Hai ngôn ngữ này hoàn toàn khác nhau.

ORM (Object-Relational Mapping) chính là một "người phiên dịch" thiên tài đứng giữa.

- Nó "nhìn" vào class Product của bạn và tự động dịch nó thành một bảng Products trong CSDL.
- Khi bạn viết một câu lệnh LINQ trong C# như `_context.Products.Where(p => p.Price > 100)`, ORM sẽ dịch nó thành câu lệnh `SELECT * FROM Products WHERE Price > 100` trong SQL.

Entity Framework Core (EF Core) là ORM phổ biến và mạnh mẽ nhất của Microsoft dành cho .NET. Nó giúp chúng ta làm việc với CSDL bằng code C# một cách tự nhiên mà không cần viết các câu lệnh SQL thủ công.

6.1.2. DbContext và DbSet

- **DbContext (ví dụ: ApplicationDbContext):** Đây là "cánh cổng" hay "phiên làm việc" của bạn với CSDL. Mọi tương tác với CSDL (đọc, ghi, xóa) đều phải thông qua một instance của DbContext. Nó theo dõi mọi thay đổi bạn thực hiện trên các đối tượng và sẽ cập nhật chúng xuống CSDL khi bạn gọi lệnh `SaveChanges()`.
- **DbSet<T> (ví dụ: DbSet<Product>):** Đại diện cho một bảng trong CSDL. Một thuộc tính kiểu `DbSet<Product>` trong DbContext cho phép bạn truy vấn và thao tác trên bảng chứa các sản phẩm.

6.1.3. Repository Pattern

Ở Buổi 2, chúng ta đã tạo ra interface `IProductRepository`. HomeController của chúng ta chỉ làm việc với interface này mà không hề biết đến FakeProductRepository. Hôm nay, chúng ta sẽ tạo một lớp mới là `EFProductRepository` cũng implement interface đó. Sau đó, chúng ta chỉ cần vào Program.cs và nói với DI Container: "Này, từ giờ đừng dùng FakeProductRepository nữa, hãy dùng EFProductRepository đi". Và BÙM, toàn bộ ứng dụng sẽ chuyển sang dùng CSDL thật mà không cần sửa một dòng code nào trong Controller. Đó chính là vẻ đẹp của việc lập trình hướng tới sự trừu tượng (interface) và giảm sự phụ thuộc.

6.2. Hướng dẫn thực hành chi tiết

6.2.1. Bước 1: Cài đặt & Cấu hình trong SportsStore.Infrastructure

Đây là project chịu trách nhiệm truy cập dữ liệu, vì vậy chúng ta sẽ thao tác chính ở đây.

1. Cài đặt các gói NuGet cần thiết:

- Chuột phải vào project SportsStore.Infrastructure -> **Manage NuGet Packages....**
- Tìm và cài đặt các gói sau (chọn phiên bản mới nhất):
 - Microsoft.EntityFrameworkCore.SqlServer (Trình điều khiển để EF Core "nói chuyện" được với SQL Server).
 - Microsoft.EntityFrameworkCore.Tools (Công cụ để quản lý CSDL, ví dụ như tạo migrations sau này).

2. Tạo ApplicationDbContext.cs:

- Trong project SportsStore.Infrastructure, tạo một class mới tên là ApplicationDbContext.cs.
- Nội dung file:

```
//C#
// SportsStore.Infrastructure/ApplicationDbContext.cs

using Microsoft.EntityFrameworkCore;
using SportsStore.Domain;

namespace SportsStore.Infrastructure;

public class ApplicationDbContext : DbContext
{
    // Constructor này nhận các tùy chọn cấu hình (như chuỗi kết nối) từ
    bên ngoài
    // thông qua Dependency Injection.
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
        : base(options) { }

    // Khai báo với EF Core rằng chúng ta có một bảng tương ứng với model
    Product.
```

```
// EF Core sẽ tự động đặt tên bảng là "Products" (dạng số nhiều).
public DbSet<Product> Products { get; set; }
}
```

3. Tạo EFProductRepository.cs:

- Đây là phiên bản "hàng thật" của Repository.
- Trong project SportsStore.Infrastructure, tạo một class mới tên là EFProductRepository.cs.
- Nội dung file:

```
//C#
// SportsStore.Infrastructure/EFProductRepository.cs
using SportsStore.Domain;
namespace SportsStore.Infrastructure;
public class EFProductRepository : IProductRepository // Implement cùng
interface với Fake repo
{
    // Inject DbContext để làm việc với CSDL
    private ApplicationDbContext _context;

    public EFProductRepository(ApplicationDbContext ctx)
    {
        _context = ctx;
    }

    // Triển khai thuộc tính Products được yêu cầu bởi interface.
    // Thay vì trả về một List hard-code, giờ đây nó trả về một DbSet.
    // EF Core sẽ dịch các truy vấn LINQ trên DbSet này thành câu lệnh SQL.
    public IQueryable<Product> Products => _context.Products;
}
```

6.2.2. Bước 2: Cấu hình trong SportsStore.WebUI

Đây là project khởi động, nơi chúng ta sẽ "dây cà ra dây muống" các thành phần lại với nhau.

1./Thêm Chuỗi kết nối vào appsettings.json:

- Mở file appsettings.json trong project SportsStore.WebUI.
- Thêm một mục ConnectionStrings như sau:

```
//JSON
{
  "Logging": {
    // ...
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "SportsStoreConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=SportsStore;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

– Giải thích chuỗi kết nối:

- Server=(localdb)\\MSSQLLocalDB: Chỉ định server CSDL là LocalDB (một phiên bản gọn nhẹ của SQL Server).
- Database=SportsStore: Đặt tên cho cơ sở dữ liệu sẽ được tạo.
- Trusted_Connection=True: Sử dụng xác thực Windows để kết nối, không cần username/password.

2./Đăng ký DbContext và Thay thế Repository trong Program.cs:

- Mở file Program.cs trong project SportsStore.WebUI.
- Thực hiện các thay đổi sau:

```
//C#
// Thêm các using cần thiết ở đầu file
using Microsoft.EntityFrameworkCore;
using SportsStore.Infrastructure;

// ... bên trong file

var builder = WebApplication.CreateBuilder(args);

// ...
builder.Services.AddControllersWithViews();

// -- BẮT ĐẦU THAY ĐỔI --

// 1. Đăng ký DbContext với DI container
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
{
    // Đọc chuỗi kết nối từ appsettings.json

    options.UseSqlServer(builder.Configuration["ConnectionStrings:SportsStoreCo
nnection"]);
});

// 2. Thay thế implementation của IProductRepository
// Dòng cũ:
// builder.Services.AddScoped<IProductRepository, FakeProductRepository>();
// Dòng mới:
builder.Services.AddScoped<IProductRepository, EFProductRepository>();

// -- KẾT THÚC THAY ĐỔI --

builder.Services.AddDistributedMemoryCache();
// ... các dịch vụ khác giữ nguyên
```

6.2.3. Bước 3: Tạo và Seeding Cơ sở dữ liệu

Code đã sẵn sàng, nhưng CSDL chưa tồn tại. Chúng ta cần một cách để tạo CSDL và điền dữ liệu ban đầu vào đó.

1. Tạo lớp SeedData.cs:

- Trong project SportsStore.Infrastructure, tạo một class static mới tên là SeedData.cs.
- Nội dung file:

```
//C#
// SportsStore.Infrastructure/SeedData.cs

using Microsoft.AspNetCore.Builder;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using SportsStore.Domain;

namespace SportsStore.Infrastructure;

public static class SeedData
```

```
{  
    public static void EnsurePopulated(IApplicationBuilder app)  
    {  
        ApplicationDbContext context = app.ApplicationServices  
            .CreateScope().ServiceProvider.GetRequiredService<ApplicationDbContext>();  
  
        // Nếu có bất kỳ migration nào đang chờ, hãy áp dụng chúng  
        if (context.Database.GetPendingMigrations().Any())  
        {  
            context.Database.Migrate();  
        }  
  
        // Chỉ seed data nếu bảng Products đang trống  
        if (!context.Products.Any())  
        {  
            // AddRange cho phép thêm nhiều đối tượng cùng lúc, hiệu quả  
            // hơn Add từng cái.  
            context.Products.AddRange(  
                new Product { Name = "Thuyền Kayak", Description = "Chiếc  
                    thuyền nhỏ cho một người chèo.", Category = "Thể thao dưới nước", Price =  
                    6600000m },  
                new Product { Name = "Áo phao cứu sinh", Description = "Bảo  
                    hộ an toàn, kiểu dáng thời trang.", Category = "Thể thao dưới nước", Price  
                    = 1175000m },  
                new Product { Name = "Bóng đá tiêu chuẩn", Description =  
                    "Bóng đạt chuẩn kích thước và trọng lượng FIFA.", Category = "Bóng đá",  
                    Price = 470000m },  
                new Product { Name = "Cờ góc sân", Description = "Tạo vẻ  
                    chuyên nghiệp cho sân bóng của bạn.", Category = "Bóng đá", Price = 840000m  
                },  
                new Product { Name = "Mô hình sân vận động", Description =  
                    "Sân vận động 35,000 chỗ ngồi (đóng gói phẳng).", Category = "Bóng đá",  
                    Price = 1908000000m },  
                new Product { Name = "Mũ Tư Duy", Description = "Cải thiện  
                    75% hiệu suất hoạt động của não.", Category = "Cờ Vua", Price = 385000m },  
                new Product { Name = "Ghế không vững", Description = "Bí  
                    mật gây bất lợi cho đối thủ của bạn.", Category = "Cờ Vua", Price = 720000m  
                },  
                new Product { Name = "Bàn cờ người", Description = "Trò  
                    chơi thú vị cho cả gia đình.", Category = "Cờ Vua", Price = 1800000m },  
                new Product { Name = "Quân Vua Kim Cương", Description =  
                    "Quân Vua mạ vàng, đính kim cương.", Category = "Cờ Vua", Price = 28800000m  
                }  
            );  
        }
```

```
},  
  
        new Product { Name = "Giày chạy bộ", Description = "Nhẹ và thoải mái cho quãng đường dài.", Category = "Chạy bộ", Price = 2400000m },  
  
        new Product { Name = "Thảm Yoga cao cấp", Description = "Bề mặt chống trượt giúp giữ thăng bằng hoàn hảo.", Category = "Fitness", Price = 840000m },  
  
        new Product { Name = "Bình nước giữ nhiệt", Description = "Giữ lạnh đồ uống trong 24 giờ.", Category = "Fitness", Price = 380000m }  
    );  
    // Quan trọng: Mọi thay đổi (Add, Update, Remove) chỉ nằm trong bộ nhớ  
  
    // cho đến khi bạn gọi SaveChanges(). Lệnh này sẽ tạo và gửi  
    // các câu lệnh INSERT tương ứng đến CSDL.  
    context.SaveChanges();  
}  
}
```

2. Gọi SeedData từ Program.cs:

- Mở lại Program.cs trong SportsStore.WebUI.
- Thêm một dòng gọi phương thức EnsurePopulated ngay trước app.Run().

```
//C#  
// ...  
app.MapControllerRoute("default",  
"{controller=Home}/{action=Index}/{id?}");  
  
// Gọi phương thức seeding dữ liệu  
SeedData.EnsurePopulated(app);  
  
app.Run();
```

6.2.4. Bước 4: Chạy và Tận hưởng thành quả!

- Nhấn **F5** để chạy ứng dụng.
- Khi ứng dụng khởi động lần đầu, phương thức EnsurePopulated sẽ được gọi. EF Core sẽ:
 1. Kiểm tra chuỗi kết nối.
 2. Thấy CSDL SportsStore chưa tồn tại -> Tự động **tạo CSDL**.

3. Thấy bảng Products chưa tồn tại -> Tự động **tạo bảng** dựa trên class Product.
 4. Thấy bảng Products đang trống -> **Thêm 5 sản phẩm** mẫu vào.
 5. Lưu các thay đổi.
- Trang web của bạn sẽ hiện ra, trông **y hệt như trước**, nhưng bây giờ tất cả dữ liệu sản phẩm, danh mục, phân trang đều đang được đọc từ CSDL SQL Server thật!

TỔNG KẾT & MỞ RỘNG

Kết quả

- **Chuyển đổi thành công:** Ứng dụng của bạn đã "tốt nghiệp" khỏi việc dùng dữ liệu giả và đang hoạt động hoàn toàn trên một CSDL thực sự.
- **Kiến trúc được chứng minh:** Bạn đã tận mắt thấy lợi ích của việc tách biệt các lớp. Toàn bộ lớp WebUI (Controllers, Views) không hề hay biết về sự thay đổi nguồn dữ liệu khổng lồ này.
- **Nền tảng vững chắc:** Giờ đây bạn đã có một nền tảng CSDL thực sự để xây dựng các tính năng phức tạp hơn như quản lý đơn hàng, xác thực người dùng.

Bài tập mở rộng

1. **Sử dụng SQL Server Object Explorer:** Trong Visual Studio, vào menu **View -> SQL Server Object Explorer**.

Tìm đến server (localdb)\MSSQLLocalDB, mở rộng mục Databases -> SportsStore -> Tables. Chuột phải vào bảng dbo.Products và chọn **View Data**. Bạn sẽ thấy chính xác 5 sản phẩm mà code của bạn đã thêm vào. Hãy thử sửa giá của một sản phẩm trực tiếp trong này và F5 lại trang web để xem sự thay đổi.
2. **Tìm hiểu về Migrations:** Phương thức context.Database.EnsureCreated() (ẩn sau logic của SeedData) rất tiện lợi nhưng không linh hoạt khi bạn muốn cập nhật schema (ví dụ thêm một cột mới). Hãy tìm hiểu về **EF Core Migrations**. Thử các lệnh sau trong **Package Manager Console** (chọn Default project là SportsStore.Infrastructure):
 - Add-Migration InitialCreate
 - Update-Database Đây là cách làm chuyên nghiệp để quản lý sự thay đổi của CSDL song song với code.

TÀI LIỆU THAM KHẢO

- [1]. **Introduction to Entity Framework Core:** [Microsoft Docs - EF Core](#)
- [2]. **Installing EF Core:** [Microsoft Docs - Installing EF Core](#)
- [3]. **DbContext Lifetime, Configuration, and Initialization:** [Microsoft Docs - DbContext Configuration](#)
- [4]. **Connection Strings:** [Microsoft Docs - Connection Strings](#)